

1 – Fundamentos da Programação

sexta-feira, 1 de maio de 2026 20:27

O que é Programação e por que Python?

1. Conceito de Programação

Definição:

Programação é o processo de criação de um programa de computador, que consiste na **implementação de um algoritmo** (sequência lógica de passos para resolver um problema) em uma linguagem de programação específica.

De forma geral, programar é **comunicar-se com o computador** por meio de uma linguagem estruturada, instruindo-o a executar operações específicas.

Analogia (importante para compreensão)

- A programação pode ser comparada a uma **receita culinária**:
- Define **ingredientes (dados)**
- Define **passos (instruções)**
- Define **ordem exata de execução**
- Assim como uma receita precisa ser detalhada, um programa também deve ser **preciso e sequencial**.

2. O que é Python?

Python é uma **linguagem de programação de alto nível**, ou seja:

Alto nível: linguagem mais próxima da linguagem humana do que da linguagem de máquina, facilitando a leitura e escrita de código.

Isso permite que o programador foque mais na **lógica do problema** do que em detalhes técnicos complexos.

3. Características do Python

3.1 Tipagem

- **Tipagem dinâmica:** o tipo da variável é definido automaticamente durante a execução, sem necessidade de declaração explícita.
- **Tipagem forte:** o Python não permite operações inválidas entre tipos diferentes sem conversão explícita.

3.2 Natureza da linguagem

- **Interpretada:** o código é executado linha por linha por um interpretador, sem necessidade de compilação prévia.

Linguagem interpretada: linguagem cujo código é traduzido para instruções de máquina **durante a execução**.

- Consequência:
- Mais **flexível**
- Mais **rápida para testar e corrigir**
- Porém, geralmente **mais lenta que linguagens compiladas** (ex: C)

3.3 Paradigma e estrutura

- **Orientada a objetos:** permite organizar o código em estruturas chamadas objetos.

Orientação a objetos: paradigma que organiza o código em entidades que combinam dados e comportamentos.

- Suporte a:
- **Módulos:** arquivos com código reutilizável
- **Pacotes:** conjunto de módulos organizados

Isso favorece:

- **Modularidade** (dividir o programa em partes)
- **Reutilização de código**

3.4 Sintaxe e produtividade

- **Sintaxe simples e legível:** facilita aprendizado e manutenção
- Foco em **legibilidade** → reduz custo de manutenção

Manutenção de software: processo de corrigir, melhorar ou adaptar um sistema ao longo do tempo.

4. Ciclo de Desenvolvimento em Python

Python não exige compilação, o que torna o ciclo de desenvolvimento mais rápido:

Etapas:

- 1. Escrever o código
- 2. Executar
- 3. Testar
- 4. Corrigir (debug)
- Esse ciclo é chamado de **edit-test-debug cycle** (ciclo de edição, teste e depuração)

5. Depuração (Debugging)

Debugging: processo de identificar e corrigir erros em um programa.

Características no Python:

- Erros geram **exceções** (mensagens de erro controladas)
- Exceção:** evento que ocorre durante a execução e interrompe o fluxo normal do programa.
- Exibe **stack trace**:
Stack trace: relatório detalhado mostrando onde o erro ocorreu no código.
- Ferramentas disponíveis:
- Depurador com:
- Inspeção de variáveis
- Execução passo a passo
- Breakpoints (pontos de parada)
- Alternativa simples:
- Uso de `print()` para acompanhar valores durante execução

6. Estruturas e Recursos do Python

- **Estruturas de dados embutidas:** listas, dicionários, conjuntos, etc.
- **Biblioteca padrão extensa:** conjunto de funcionalidades prontas
- **Distribuição gratuita e multiplataforma**

7. Vantagens do Python

- Fácil aprendizado
- Alta produtividade
- Grande comunidade global
- Excelente documentação
- Uso amplo em:
- Indústria
- Pesquisa acadêmica
- Educação

8. Evolução e História do Python

- Criado em **1989** por **Guido van Rossum**
- Lançado em **1991**
- Evoluiu como uma linguagem:
- Simples
- Legível
- Poderosa

9. Comparação: Linguagens Interpretadas vs Compiladas

Característica	Interpretadas (Python)	Compiladas (C)
Execução	Interpretado/traduzido enquanto executa	Traduz tudo para em seguida executar
Velocidade	Menor	Maior
Flexibilidade	Maior	Menor
Facilidade de testes	Alta	Menor

Termos/Assuntos que pedi para a ia explicar para mim:

Analogia MUITO simples (isso aqui costuma desbloquear)

 Python (interpretado)

Você está cozinhando:

- lê a receita
- faz o passo 1
- depois lê o passo 2

- faz o passo 2

👉 Tudo ao mesmo tempo

📁 Compilado

Você:

- lê TODA a receita primeiro
- prepara tudo
- depois começa a cozinhar

1. Tipagem dinâmica (o mais importante de entender)

Significa que você **NÃO precisa dizer o tipo da variável**.

Exemplo em Python:

```
x = 10      # inteiro
x = "oi"    # agora virou texto
```

👉 O Python aceita isso de boa.

Em outras linguagens (tipo C ou Java):

Você teria que declarar antes:

```
int x = 10;
```

Resumindo:

- **Tipagem dinâmica = o Python descobre o tipo sozinho em tempo real**

2. Tipagem forte

Significa que o Python **não deixa misturar coisas sem sentido**.

Exemplo:

```
print("10" + 5)
```

❌ Isso dá erro

Porque:

- "10" é texto
- 5 é número

O Python fala:

👉 "Não vou somar isso não, não faz sentido"

Mas se você quiser:

```
print(int("10") + 5)
```

✅ Ai funciona (porque você converteu)

Resumindo:

- **Tipagem forte = o Python evita misturar tipos incompatíveis**

Termos importantes

- **Programação:** criação de programas por meio de instruções para o computador.
- **Algoritmo:** sequência lógica de passos para resolver um problema.
- **Linguagem de alto nível:** linguagem próxima da linguagem humana.
- **Tipagem dinâmica:** tipos definidos automaticamente em tempo de execução.
- **Tipagem forte:** impede operações inválidas entre tipos.
- **Interpretador:** programa que executa o código linha por linha.
- **Compilação:** tradução do código antes da execução.
- **Orientação a objetos:** organização do código em objetos.
- **Módulo:** arquivo com código reutilizável.
- **Pacote:** conjunto de módulos.
- **Debugging:** processo de encontrar e corrigir erros.
- **Exceção:** erro tratado durante a execução.
- **Stack trace:** detalhamento do erro no código.
- **Biblioteca padrão:** conjunto de funções prontas da linguagem.
- **Modularidade:** divisão do código em partes reutilizáveis.

=====

Variáveis: O Coração do Armazenamento de Dados

Conceito de Variáveis

Definição: Uma variável é um espaço na memória do computador utilizado para **armazenar valores** que podem ser utilizados e manipulados durante a execução de um programa.

- Pode ser comparada a uma **caixa etiquetada**:
- **Nome da variável** → rótulo da caixa
- **Valor** → conteúdo armazenado

Segundo Manzano e Oliveira (2020, p. 45):

“As variáveis são os elementos fundamentais para o desenvolvimento de qualquer programa, pois permitem armazenar valores que serão manipulados durante a execução do programa”.

Importância das Variáveis

As variáveis são utilizadas principalmente por dois motivos:

1. Flexibilidade

Permite alterar valores facilmente sem modificar várias partes do código.

2. Reutilização

Evita repetição de valores no código.

Exemplo:

- Em vez de escrever várias vezes o número 10, você pode fazer:
`valor = 10`

E usar `valor` sempre que necessário.

Declaração de Variáveis em Python

Em Python, declarar uma variável é simples:

```
idade = 25
nome = "João"
altura = 1.75
```

Características importantes:

- Não é necessário declarar o tipo explicitamente.
- O Python identifica automaticamente o tipo do valor.

Tipagem Dinâmica

Tipagem dinâmica: mecanismo em que o Python **detecta automaticamente o tipo de dado** armazenado na variável.

Exemplo:

- `idade` → inteiro (**int**)
- `nome` → texto (**string**)
- `altura` → número decimal (**float**)

Vantagem: facilita a escrita do código.

Desvantagem: exige atenção do programador para evitar erros com tipos de dados.

Entrada e Saída de Dados

Função `input()`

`input()`: função utilizada para **capturar dados do usuário** via teclado.

Função `print()`

`print()`: função utilizada para **exibir informações na tela**.

Exemplo Prático

Objetivo:

Criar um programa que:

- Solicite nome e idade do usuário
- Armazene em variáveis
- Exiba uma mensagem personalizada

Exemplo com concatenação:

```
nome = input("Digite seu nome: ")
idade = input("Digite sua idade: ")
print("Olá,", nome, "! Você tem", idade, "anos.")
```

Exemplo com f-string (forma mais moderna):

```
nome = input('Digite seu nome: ')
idade = input('Digite sua idade: ')
print(f'Olá, {nome}! Você tem {idade} anos.')
```

f-string: forma de formatação de strings que permite inserir variáveis diretamente dentro de textos usando `{}`.

Convenções de Nomenclatura em Python

Regras básicas:

- Deve começar com:
- letra (a-z, A-Z) ou `_` (underscore)
- Não pode começar com número
- Pode conter:
- letras, números e `_`
- Diferencia maiúsculas de minúsculas (**case sensitive**)
- Não pode usar palavras reservadas

Padrão PEP 8 (Guia Oficial de Estilo)

Fonte: Documentação oficial Python – PEP 8

🔗 <https://peps.python.org/pep-0008/>

Objetivo:

Padronizar o código para melhorar **legibilidade** e **consistência**.

Princípios importantes:

- “**Legibilidade é fundamental**”
- Código é mais lido do que escrito
- Consistência dentro do projeto é essencial

Convenções de Nomes (PEP 8)

1. Variáveis e funções

- Usar **snake_case**:
`peso_ideal`
`numero_tentativas`
`taxa_conversao`

2. Classes

- Usar **CamelCase (CapWords)**:
`MinhaClasse`
`UsuarioSistema`

3. Constantes

- Letras maiúsculas com underscore:
`MAX_VALOR`
`TOTAL_USUARIOS`

4. Nomes especiais com underscore

- `_variavel` → uso interno
- `variavel_` → evitar conflito com palavra reservada
- `__variavel` → evita conflitos em herança
- `__init__` → método especial (mágico)

Boas Práticas de Código (PEP 8)

Indentação

- Usar **4 espaços** por nível
- Não misturar espaços e tabulações

Comprimento de linha

- Máximo recomendado: **79 caracteres**

Espaçamento

- Evitar espaços desnecessários:

Correto
`x = 1`

Incorreto
`x = 1`

Importações

- Devem ficar no início do arquivo
- Uma por linha:

```
import os
import sys
```

Comentários

- Devem ser claros e atualizados
- Usar frases completas
- Evitar comentários óbvios

Boas práticas gerais

- Usar `is` para comparar com `None`
`if x is None:`
- Usar `isinstance()` para verificar tipos:
`isinstance(x, int)`
- Evitar comparar com `True` ou `False` diretamente:
`if variavel: # correto`

Termos importantes

- **Variável:** espaço na memória que armazena dados.
- **Tipagem dinâmica:** o tipo da variável é definido automaticamente pelo Python.
- **Input:** função para entrada de dados do usuário.
- **Print:** função para saída de dados.
- **String:** sequência de caracteres (texto).
- **Int:** número inteiro.
- **Float:** número decimal.
- **f-string:** forma moderna de formatar strings com variáveis.
- **Snake_case:** padrão de nomes com letras minúsculas e underscore.
- **PEP 8:** guia oficial de estilo para código Python.
- **Indentação:** organização do código por espaçamento.
- **Constante:** valor fixo que não deve ser alterado.

=====

Tipos de Dados Primitivos em Python: Os Blocos Construtivos

Visão Geral

Python possui **tipos de dados primitivos**, que são os elementos básicos utilizados para armazenar e manipular informações em um programa. Compreender esses tipos é essencial para escrever códigos corretos e eficientes.

Números Inteiros (int)

Definição: Tipo de dado que representa números sem casas decimais.

- Podem ser **positivos, negativos ou zero**
- Exemplos: -15, 0, 42, 1000000
- Usos comuns:
- Contagens
- Índices de listas
- Operações matemáticas sem necessidade de decimais

Números Reais (float)

Definição: Tipo de dado que representa números com casas decimais.

- Exemplos: 3.14, -2.5, 0.0, 1.999
- Características:
- Resultados de operações matemáticas com decimais retornam **float**
- Possuem **limitações de precisão** devido à representação em binário
- Usos comuns:
- Medidas
- Valores monetários
- Cálculos científicos

Texto (str)

Definição: Tipo de dado que representa uma sequência de caracteres (cadeia de texto).

- Criado com **aspas simples (' ') ou duplas (" ")**
- Exemplos: "Olá, mundo", 'Programação', "123"
- Importante:
- "123" é **texto**, não número

- Operações matemáticas não funcionam da mesma forma
- Usos comuns:
- Nomes
- Mensagens
- Manipulação de texto

Valores Lógicos (bool)

Definição: Tipo de dado que representa valores de verdade.

- Possui apenas dois valores:
- **True** (verdadeiro)
- **False** (falso)
- Usos comuns:
- Controle de fluxo do programa
- Estruturas condicionais (**if**, **while**, **for**)

Comparativo dos Tipos Primitivos

Tipo	Exemplo	Uso comum
int	42	Contagens, índices
float	3.14	Cálculos decimais, medidas
str	"Olá"	Textos, mensagens
bool	True/False	Decisões lógicas

Uso dos Tipos no Python

- **int** e **float** → utilizados em operações matemáticas
- **str** → manipulação de dados textuais
- **bool** → controle de decisões no programa

Importância da Separação dos Tipos

A existência de diferentes tipos garante:

- **Eficiência:** inteiros ocupam menos memória que floats
- **Precisão:** operações são mais exatas quando o tipo é adequado
- **Clareza:** facilita entender o que cada dado representa

Conversão de Tipos (Automática)

Python pode converter tipos automaticamente em algumas operações.

Exemplo:

- Preço: 19.99 (**float**)
- Quantidade: 5 (**int**)
- Resultado: 99.95 (**float**)

Isso ocorre porque Python realiza uma **conversão implícita** (*transformação automática de tipos durante operações*).

Exemplo Prático

Cálculo de custo total de uma compra:

- Preço unitário: 19.99 (**float**)
- Quantidade: 5 (**int**)
- Resultado final: 99.95 (**float**)

Reflexão: Escolha do Tipo de Dado

Exemplo: Sistema Bancário

8. **Saldo da conta** → float (ou biblioteca específica para maior precisão)
9. **Nome do titular** → str
10. **Conta ativa/inativa** → bool

Por que isso importa?

A escolha correta do tipo impacta:

- **Precisão** (evitar erros de arredondamento)
 - **Performance** (uso eficiente de memória)
 - **Confiabilidade** (evitar falhas no sistema)
- Situação crítica
- Em sistemas bancários ou hospitalares:

- Usar **float para dinheiro** pode gerar erros de arredondamento
- O ideal é usar **bibliotecas específicas para valores monetários**

Termos Importantes

- **Tipo de dado primitivo**: categoria básica de dados usada para armazenar valores simples.
 - **int**: número inteiro sem casas decimais.
 - **float**: número com casas decimais.
 - **str (string)**: sequência de caracteres (texto).
 - **bool**: valor lógico (True ou False).
 - **Conversão implícita**: transformação automática de um tipo em outro durante operações.
 - **Precisão numérica**: nível de exatidão de um número armazenado no computador.
 - **Representação binária**: forma como os dados são armazenados usando 0 e 1 no computador.
- =====

Operadores em Python: Transformando e Comparando Dados

1. Conceito de Operadores

Operadores: símbolos que indicam que uma operação deve ser realizada sobre um ou mais **operandos** (dados utilizados na operação).

Python possui diferentes categorias de operadores, cada uma com funções específicas no processamento de dados.

2. Operadores Aritméticos

Definição: operadores utilizados para realizar operações matemáticas básicas, fundamentais em programas que manipulam dados numéricos.

Principais operadores:

- **Adição (+)**
Exemplo: $10 + 5 \rightarrow 15$
- **Subtração (-)**
Exemplo: $10 - 5 \rightarrow 5$
- **Multiplicação (*)**
Exemplo: $10 * 5 \rightarrow 50$
- **Divisão (/)**
Exemplo: $10 / 5 \rightarrow 2.0$

Observação: sempre retorna um número do tipo **float** (número com casas decimais).

- **Divisão inteira (//)**
Exemplo: $10 // 3 \rightarrow 3$

Divisão inteira: descarta as casas decimais.

- **Módulo (%)**
Exemplo: $10 \% 3 \rightarrow 1$
Módulo: retorna o resto da divisão.
- **Potência (**)**
Exemplo: $2 ** 3 \rightarrow 8$

Ordem de precedência:

Python segue a ordem matemática padrão:

11. Parênteses
12. Potência
13. Multiplicação e divisão
14. Adição e subtração

Observação: parênteses podem ser usados para alterar a ordem das operações.

3. Operadores Relacionais

Definição: operadores que comparam dois valores e retornam um valor booleano (**True** ou **False**).

Principais operadores:

- **Igual (==)**
 $5 == 5 \rightarrow \text{True}$
- **Diferente (!=)**
 $5 != 3 \rightarrow \text{True}$
- **Maior que (>)**
 $5 > 3 \rightarrow \text{True}$
- **Menor que (<)**
 $5 < 3 \rightarrow \text{False}$
- **Maior ou igual (>=)**
 $5 >= 5 \rightarrow \text{True}$
- **Menor ou igual (<=)**
 $3 <= 5 \rightarrow \text{True}$

Importância: são essenciais para estruturas de controle, como condicionais (if, else).

4. Operadores Lógicos

Definição: operadores que combinam valores booleanos para gerar novos resultados booleanos.

Principais operadores:

- **and (E)**
Retorna **True** apenas se ambas as condições forem verdadeiras.
Exemplo: $(5 > 3) \text{ and } (3 > 1) \rightarrow \text{True}$
- **or (OU)**
Retorna **True** se pelo menos uma condição for verdadeira.
Exemplo: $(5 > 10) \text{ or } (3 > 1) \rightarrow \text{True}$
- **not (NÃO)**
Inverte o valor booleano.
Exemplo: $\text{not } (5 > 10) \rightarrow \text{True}$

5. Operadores de Atribuição Compostos

Definição: operadores que combinam uma operação matemática com a atribuição de valor a uma variável, tornando o código mais simples e legível.

Atribuição básica:

- $= \rightarrow$ atribui um valor a uma variável
Exemplo: $x = 10$

Operadores compostos:

- $+=$ (adição com atribuição)
 $x += 5 \rightarrow$ equivale a $x = x + 5$
- $-=$ (subtração com atribuição)
 $x -= 5 \rightarrow$ equivale a $x = x - 5$
- $*=$ (multiplicação com atribuição)
 $x *= 2 \rightarrow$ equivale a $x = x * 2$
- $//=$ (divisão inteira com atribuição)
- $\%=$ (módulo com atribuição)
- $=$ (potência com atribuição)

Vantagem: deixa o código mais enxuto e facilita operações repetitivas.

6. Exercício Prático: Soma de Números

Objetivo:

Criar um programa que:

15. Solicite números ao usuário.
16. Some todos os valores digitados.
17. Encerre quando o usuário digitar 0.
18. Exiba a soma total.

Código:

```
soma = 0
while True:
    numero = int(input("Digite um número (0 para sair): "))
    if numero == 0:
        break
    soma += numero
print("A soma dos números digitados é:", soma)
```

Explicação do processo:

19. Inicializa a variável **soma = 0**
20. Entra em um loop infinito (**while True**)
21. Recebe um número do usuário
22. Verifica se o número é 0:
 - Se for $\rightarrow$ encerra o programa (**break**)
 - Se não $\rightarrow$ adiciona à soma (**soma += numero**)
23. Exibe o resultado final

Dica de aprimoramento:

- Contar quantos números foram digitados
- Calcular a média dos valores

Termos importantes

- **Operador:** símbolo que realiza operações sobre dados.
- **Operando:** valor sobre o qual o operador atua.
- **Float:** número com casas decimais.
- **Divisão inteira:** divisão que descarta a parte decimal.
- **Módulo (%):** resto de uma divisão.
- **Booleano:** tipo de dado que pode ser **True** ou **False**.

- **Operadores relacionais:** usados para comparação entre valores.
 - **Operadores lógicos:** combinam expressões booleanas.
 - **Atribuição:** processo de armazenar valores em variáveis.
 - **Atribuição composta:** combinação de operação + atribuição.
 - **Estrutura de controle:** comandos que controlam o fluxo do programa (ex: if, while).
 - **Loop (laço de repetição):** estrutura que repete instruções (ex: while).
- =====

Entrada e Saída de Dados: Comunicação com o Usuário

1. Importância da Entrada e Saída de Dados

Um programa que não recebe dados nem exibe resultados possui **utilidade limitada**.

Por isso, é essencial estabelecer **comunicação com o usuário**, permitindo:

- Receber informações (entrada)
- Exibir resultados (saída)

2. Saída de Dados com `print()`

A função `print()` é utilizada para exibir informações na tela.

Características principais:

- Pode exibir textos, números e variáveis
- Aceita múltiplos argumentos separados por vírgula
- Por padrão, separa os valores com **espaço**

Exemplos:

```
print("Olá, mundo!")
print(42)
print("A resposta é:", 42)
```

```
resultado = 10 + 5
print("10 + 5 =", resultado)
```

Resumo:

- **Função `print()`:** exibe dados na tela (saída de dados)
- Fundamental para entender o funcionamento do programa

3. Entrada de Dados com `input()`

A função `input()` permite receber dados do usuário.

Funcionamento:

- Pausa o programa
- Aguarda o usuário digitar algo
- Retorna o valor digitado como **string**

Exemplo:

```
nome = input("Qual é o seu nome? ")
print("Bem-vindo,", nome)
```

Ponto importante:

- **`input()` sempre retorna uma string**, independentemente do valor digitado

4. Conversão de Tipos (Type Casting)

Conversão de tipos: transformação de um valor de um tipo para outro.

Principais funções:

- **`int()`:** converte para número inteiro
Exemplo: `int("25")` → 25
⚠ `int("25.5")` gera erro
- **`float()`:** converte para número real (decimal)
Exemplo: `float("25.5")` → 25.5
- **`str()`:** converte para string
Exemplo: `str(42)` → "42"
- **`bool()`:** converte para booleano (verdadeiro ou falso)
Exemplo:
• `bool(1)` → True
• `bool(0)` → False

5. Tratamento de Entrada do Usuário

Ao solicitar dados numéricos com `input()`, é necessário converter para o tipo adequado.

Exemplo comum:

```
idade = int(input("Qual é sua idade? "))
```

Problema:

Se o usuário digitar algo inválido (ex: "abc"), ocorre um erro.

Observação importante:

- É necessário validar a entrada do usuário
- Programas mais robustos utilizam **tratamento de exceções**

6. Exemplo Prático: Cálculo de IMC**Código:**

```
nome = input("Digite seu nome: ")
peso = float(input("Digite seu peso em kg: "))
altura = float(input("Digite sua altura em metros: "))

imc = peso / (altura ** 2)

print(nome + ", seu IMC é:", imc)

if imc < 18.5:
    print("Você está abaixo do peso.")
elif imc < 25:
    print("Você está com peso normal.")
else:
    print("Você está acima do peso.")
```

Conceitos aplicados:

- Entrada de dados (input)
- Conversão de tipos (float)
- Cálculo matemático
- Estrutura condicional (if, elif, else)

7. Tratamento de Exceções (try / except)

O **tratamento de exceções** evita que o programa pare ao ocorrer um erro.

Definição:

Tratamento de exceções: mecanismo para lidar com erros durante a execução do programa.

Estrutura:

```
try:
    # código que pode gerar erro
except TipoDoErro:
    # tratamento do erro
```

Significado:

- **try (tente):** tenta executar o código
- **except (caso ocorra erro):** executa se um erro acontecer

Exemplo:

```
soma = 0
while True:
    try:
        numero = int(input("Digite um número (0 para sair): "))

        if numero == 0:
            break

        soma += numero

    except ValueError:
        print("Entrada inválida! Por favor, digite apenas números inteiros.")
```

Explicação:

24. O programa tenta converter a entrada para inteiro (try)
25. Se o usuário digitar algo inválido:
 - O erro **ValueError** é capturado
 - O programa não encerra
 - Exibe uma mensagem de erro

8. Operador de Atribuição Composto

- **+=**: soma e atribui ao mesmo tempo

Exemplo:

```
soma += numero
```

Equivalente a:

```
soma = soma + numero
```

Termos Importantes

- **Entrada de dados**: informações fornecidas pelo usuário ao programa
- **Saída de dados**: informações exibidas pelo programa
- **print()**: função que exibe dados na tela
- **input()**: função que recebe dados do usuário (retorna string)
- **Conversão de tipos**: transformação de um valor para outro tipo (int, float, etc.)
- **int()**: converte para inteiro
- **float()**: converte para número decimal
- **str()**: converte para texto
- **bool()**: converte para verdadeiro ou falso
- **Tratamento de exceções**: técnica para lidar com erros no programa
- **try**: tenta executar um código
- **except**: executa caso ocorra erro
- **ValueError**: erro quando a conversão de tipo falha
- **Operador +=**: soma e atribui valor à variável

=====

Bibliotecas Padrão em Python: Expandindo as Capacidades

Conceito Geral

Biblioteca-padrão: conjunto de módulos (arquivos com funções e recursos prontos) que já vêm incluídos no Python, permitindo reutilizar funcionalidades sem precisar programar tudo do zero.

Importação de módulos: processo de incluir uma biblioteca no programa usando a palavra-chave `import`.

Padrão de uso:

26. Importar o módulo
27. Acessar funções/constantes com `nome_do_modulo.funcao()`

Biblioteca math

Função

Fornece **operações matemáticas avançadas** e constantes.

Importação

```
import math
```

Principais recursos

- **math.pi**: constante do valor de π (pi)
- **math.sqrt(x)**: calcula a **raiz quadrada**
- **math.ceil(x)**: arredonda para cima
- **math.floor(x)**: arredonda para baixo
- **math.factorial(x)**: calcula o **fatorial** (multiplicação de todos os inteiros positivos até x)

Exemplos

```
print(math.pi)           # 3.141592653589793
print(math.sqrt(25))     # 5.0
print(math.ceil(4.3))    # 5
print(math.floor(4.9))   # 4
print(math.factorial(5))  # 120
```

Biblioteca random

Função

Responsável por gerar **valores aleatórios**, muito usada em jogos, simulações e sorteios.

Importação

```
import random
```

Principais recursos

- **random.randint(a, b)**: número inteiro aleatório entre a e b
- **random.random()**: número decimal entre 0 e 1
- **random.choice(lista)**: seleciona um elemento aleatório de uma lista
- **random.shuffle(lista)**: embaralha os elementos da lista

Exemplos

```
numero_aleatorio = random.randint(1, 100)
valor_decimal = random.random()
lista = [1, 2, 3, 4, 5]
elemento = random.choice(lista)
random.shuffle(lista)
```

Biblioteca `datetime`

Função

Manipula **datas e horários**, sendo útil para registros, prazos e cálculos temporais.

Importação

```
from datetime import datetime, date
```

Principais recursos

- **`datetime.now()`**: retorna data e hora atuais
- **`date.today()`**: retorna a data atual
- **`.year / .month`**: acessa partes da data
- Criação de datas: `date(ano, mês, dia)`

Exemplos

```
agora = datetime.now()
print(agora)
```

```
hoje = date.today()
print(hoje.year, hoje.month)
```

```
data_nascimento = date(1990, 5, 15)
idade = hoje.year - data_nascimento.year
```

Resumo Comparativo das Bibliotecas

Biblioteca	Função Principal	Exemplos
math	Operações matemáticas	<code>math.sqrt(25)</code>
random	Geração de valores aleatórios	<code>random.randint(1,10)</code>
datetime	Manipulação de datas e horas	<code>datetime.now()</code>

Programa Prático: Gerador de Números da Sorte

Objetivo

Combinar conceitos de:

- Entrada e saída de dados
- Tipos de dados
- Operadores
- Uso de biblioteca (`random`)

Código

```
import random
```

```
print("=" * 40)
print("GERADOR DE NÚMEROS DA SORTE")
print("=" * 40)
```

```
nome = input("\nQual é o seu nome? ")
mes = int(input("Em qual mês você nasceu? (1-12) "))
dia = int(input("Em qual dia você nasceu? (1-31) "))
```

```
# Gera 6 números aleatórios entre 1 e 60
numeros_sorte = sorted(random.sample(range(1, 61), 6))
```

```
# Calcula um número especial baseado na data
numero_especial = (mes + dia) % 10
```

```
print("\n" + "-" * 40)
print(f"Olá, {nome}!")
print(f"Seus números da sorte são: {numeros_sorte}")
print(f"Seu número especial é: {numero_especial}")
print("-" * 40)
```

Explicação dos principais conceitos

- **`input()`**: função que recebe dados do usuário
- **`int()`**: converte texto para número inteiro
- **`random.sample()`**: seleciona múltiplos elementos únicos aleatórios
- **`range(início, fim)`**: gera sequência de números

- **sorted()**: ordena uma lista
- **% (módulo)**: retorna o resto da divisão (usado para limitar valores)

Termos importantes

- **Biblioteca-padrão**: conjunto de módulos já incluídos no Python com funções prontas.
- **Módulo**: arquivo que contém funções e variáveis reutilizáveis.
- **Importação (import)**: mecanismo para acessar bibliotecas externas ou internas.
- **Função**: bloco de código reutilizável que executa uma tarefa específica.
- **Constante**: valor fixo que não muda durante a execução do programa (ex: π).
- **Aleatoriedade**: geração de valores imprevisíveis.
- **Data e hora (datetime)**: representação de informações temporais.
- **Fatorial**: produto de todos os números inteiros positivos até um número.
- **Módulo (%)**: operador que retorna o resto de uma divisão.

=====

Boas Práticas desde o Início (Programação em Python)

Importância das Boas Práticas

Desenvolver bons hábitos desde o início é essencial para escrever código de qualidade.

- A forma como o código é escrito influencia:
- A **manutenção futura**
- O **trabalho em equipe**
- A **legibilidade**

Princípio importante:

Código claro é melhor que código criativo (Ramalho, 2023).

Comentários no Código

O que são comentários?

Comentários: anotações no código que **não são executadas pelo computador**, usadas para explicar partes do programa.

- Em Python, começam com #

Exemplo:

```
# Isto é um comentário
preco_unitario = 19.99 # Preço do produto em reais
quantidade = 5
```

```
# Calculamos o total multiplicando preço pela quantidade
total = preco_unitario * quantidade
print(f"Total: R$ {total:.2f}")
```

Boas práticas com comentários:

- Devem explicar **por que** algo foi feito
- Evitar explicar **o que já é óbvio no código**

❌ Exemplo ruim:

```
x = 5 # Atribui 5 a x
```

✓ Melhor prática:

- Tornar o código claro o suficiente para reduzir a necessidade de comentários

Legibilidade do Código

Legibilidade: facilidade com que humanos conseguem ler e entender o código.

Por que é importante?

- Facilita manutenção
- Ajuda no trabalho em equipe
- Melhora o aprendizado

Formatação de Código

Exemplo de código mal formatado:

```
x=5;print(x)
```

Problemas:

- Falta de espaçamento
- Uso desnecessário de ;
- Dificuldade de leitura

Exemplo de código bem formatado:

```
x = 5
print(x)
```

Vantagens:

- Espaçamento adequado melhora a leitura
- Cada instrução em uma linha

Nomenclatura de Variáveis

Nomenclatura: forma de nomear variáveis e elementos do código.

Boas práticas:

- Usar nomes **descritivos**
- Tornar o código **autoexplicativo**

✓ Exemplo:

peso_usuario

✗ Evitar:

p
x

Espaçamento e Organização

- A formatação **não altera a execução**, mas impacta muito a leitura
- Separar blocos de código com linhas em branco
- Usar indentação correta

Indentação: organização do código em níveis usando espaços no início da linha, essencial em Python para definir blocos de código.

PEP 8 – Guia de Estilo Python

PEP 8 (Python Enhancement Proposal 8): guia oficial de estilo para código Python, que define padrões para tornar o código mais legível e consistente.

Principais regras:

28. Usar **4 espaços** para indentação (não usar tabs)
29. Limitar linhas a **79 caracteres**
30. Separar funções com **duas linhas em branco**
31. Usar **snake_case** para nomes de variáveis

snake_case: padrão de nomenclatura onde palavras são separadas por underscore (`_`), como em `nome_usuario`.

Dica Geral

Seguir boas práticas:

- Evita erros
- Facilita entendimento
- Torna o código mais profissional
- Ajuda outras pessoas (e você no futuro) a compreender o código

Termos importantes

- **Comentário:** anotação no código que não é executada, usada para explicar decisões.
- **Legibilidade:** facilidade de leitura e compreensão do código.
- **Nomenclatura:** forma de nomear variáveis e elementos.
- **Indentação:** uso de espaços para organizar blocos de código.
- **PEP 8:** guia oficial de estilo do Python.
- **snake_case:** padrão de nomes com palavras separadas por `_`.
- **Formatação de código:** organização visual do código para facilitar leitura.

Depuração: Encontrando e Corrigindo Erros

Conceito de Depuração

Depuração (Debugging): processo de **identificar, analisar e corrigir erros** em um programa.

A depuração é uma habilidade fundamental na programação, sendo tão importante quanto escrever código corretamente.

Definição: Depuração é a capacidade de encontrar e corrigir erros de forma eficiente durante o desenvolvimento de software.

- Todo(a) programador(a), iniciante ou experiente, enfrenta erros.

- Erros são **inevitáveis** e fazem parte do aprendizado.
- Desenvolver habilidade em depuração diferencia programadores iniciantes de programadores mais competentes.

Tipos de Erros em Python

1. Erros de Sintaxe

Erro de sintaxe: ocorre quando o código viola as regras gramaticais da linguagem Python.

Características:

- O interpretador interrompe a execução do programa.
- O erro é indicado com **localização exata**.
- Geralmente são mais fáceis de corrigir.

Exemplos:

```
print("Olá # Falta aspas de fechamento
```

```
nome = João # João deveria estar entre aspas: "João"
```

2. Erros de Lógica

Erro de lógica: ocorre quando o código está **sintaticamente correto**, mas o resultado está incorreto.

Características:

- O programa executa normalmente.
- Não há mensagens de erro.
- O resultado não corresponde ao esperado.
- Mais difíceis de identificar.

Exemplo:

```
# Programa para calcular a média
nota1 = 8
nota2 = 7
media = nota1 + nota2 # ERRO: deveria ser (nota1 + nota2) / 2
print(media) # Exibe 15 em vez de 7.5
```

Técnicas Básicas de Depuração

1. Uso estratégico do print()

print(): função usada para exibir informações no console.

Como usar na depuração:

- Inserir prints em pontos críticos do código.
- Verificar valores de variáveis durante a execução.
- Acompanhar o fluxo do programa.

Objetivo: identificar onde os valores começam a ficar incorretos.

2. Testar seções pequenas

Divisão do problema: técnica que consiste em separar o código em partes menores.

Etapas:

32. Dividir o programa em blocos.
33. Testar cada bloco separadamente.
34. Identificar qual parte apresenta erro.

Vantagem: facilita a localização do problema.

3. Repensar a lógica

Quando o programa funciona, mas o resultado está errado:

Estratégias:

- Revisar a sequência de passos.
- Escrever o algoritmo em linguagem natural.
- Criar diagramas (fluxogramas).
- Testar manualmente com exemplos simples.

Objetivo: entender onde o raciocínio lógico falhou.

Desenvolvimento de Resiliência Técnica

Resiliência técnica: capacidade de lidar com erros e dificuldades sem desistir.

- Erros são parte essencial do aprendizado.

- Cada erro ajuda a compreender melhor:
- O funcionamento da linguagem.
- Técnicas de depuração.
- Estratégias de resolução de problemas.

Reflexão:

- Como você reage aos erros?
- É possível encará-los como oportunidades de aprendizado?
- Uma mudança de mentalidade pode melhorar sua evolução na programação.

Termos importantes

- **Depuração (Debugging)**: processo de encontrar e corrigir erros no código.
- **Erro de sintaxe**: erro causado por violação das regras da linguagem.
- **Erro de lógica**: erro em que o código funciona, mas produz resultado incorreto.
- **Interpretador**: programa que executa o código Python linha por linha.
- **print()**: função usada para exibir dados no console.
- **Fluxo do programa**: ordem em que as instruções são executadas.
- **Algoritmo**: sequência lógica de passos para resolver um problema.
- **Resiliência técnica**: habilidade de lidar com erros de forma produtiva e persistente.

=====